

AI-Assisted Mainframe Modernization

Using Spec Kit + Agents of Agents to renovate a public COBOL/CICS health-claim sample into a PBM Rx adjudication modernization demo.

Legacy reference: z/OS Connect COBOL sample

Modern target: Node.js + Python validation

Governed by UAT + compliance evidence

Core idea

Modernize by business capability — not by blindly translating COBOL.

High-level walkthrough deck

STEP 1

Project setup: keep legacy safe, generate modern code separately

legacy-reference/
Public z/OS Connect COBOL health-claim sample.
Used as the legacy baseline and golden master.

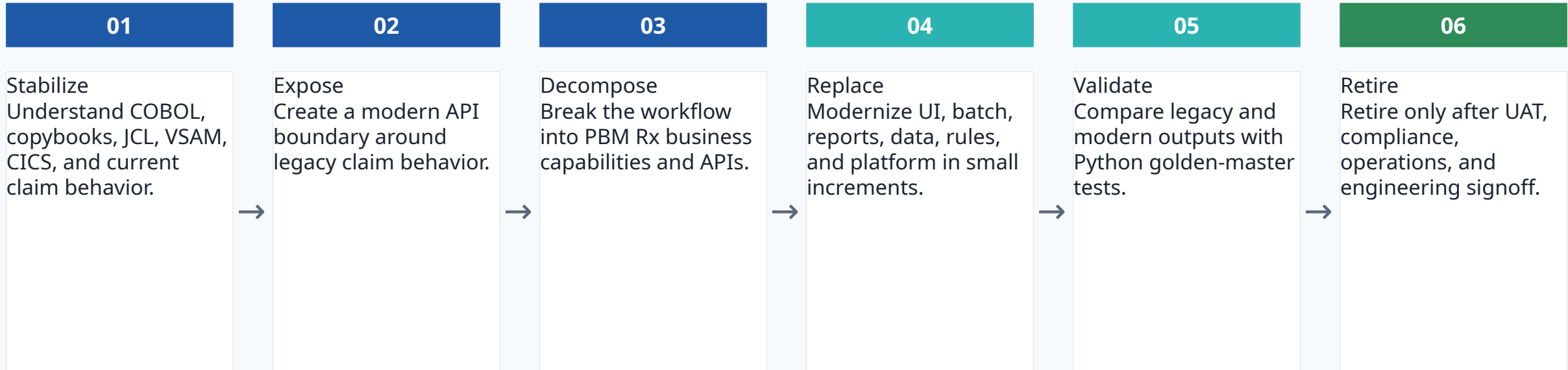
agents/
Role files guide discovery, decomposition, API design,
risk, coding, and platform work.

style-reference/
Node.js and Python examples define the coding style
to inherit.

modern outputs/
Generated code goes to modern-node-api, python-
validation, dashboard, docs, specs, platform.

Rule: do not overwrite the legacy-reference folder.

Modernization sequence



This creates a controlled path from legacy evidence to selective retirement.

Agents of Agents: divide the work before coding

Understand legacy

- COBOL Discovery
- Business Rule
- AI Modernization
- Capability Decomposition

Design target

- API Design
- Data Modernization
- UI Modernization
- Batch Modernization

Build & govern

- Developer
- Test Generation
- Risk & Compliance
- Platform Engineering

Operating rule

Discovery, capability decomposition, data mapping, and risk review happen before implementation.

Decompose the claim flow into replaceable capabilities

Rx claim intake

Eligibility check

Formulary check

Cost threshold

Prior authorization

**Adjudication
decision**

Pending review

Audit & reporting

Each capability gets its own API, tests, data model, risk profile, and replacement plan.

Target architecture: modern layers around legacy core

Modern Web / Mobile UI
React dashboard, workflow views, rule explanations



Node.js API / BFF Layer
REST APIs, DTOs, service layer, mock mainframe adapter



Workflow + Rules Layer
Temporal-ready orchestration, rules service, audit events



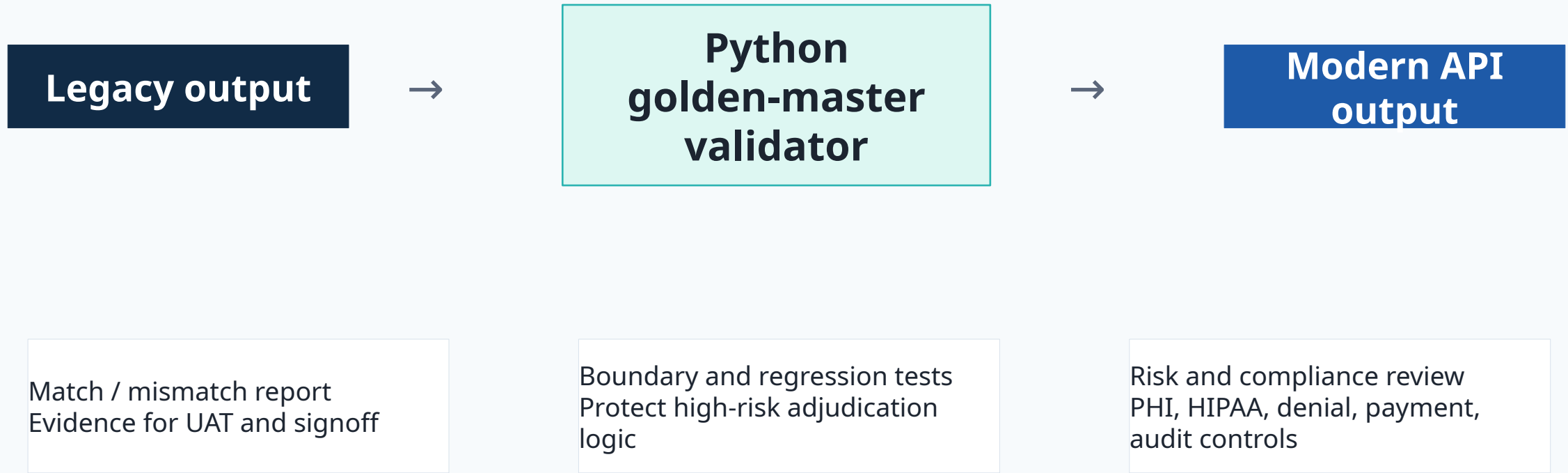
Validation + Analytics Layer
Python golden master, reports, Fabric / Databricks / BI-ready data



Legacy Mainframe Core
COBOL / CICS / z/OS Connect / DB2 or VSAM reference behavior

The legacy system stays stable while modern capabilities grow around it.

Parallel validation turns migration into evidence



No replacement without evidence.

Demo flow: what to show in 2 minutes

1

Show the folders
legacy-reference, agents, style-reference, modern-node-api, python-validation, docs

2

Explain the strategy
stabilize → expose → decompose → replace → validate → retire

3

Show agent roles
specialized agents for COBOL, rules, API, data, UI, batch, AI, risk, platform

4

Show the target
Node.js API + Python validation + dashboard + governance documents

5

End with the value
evidence-based modernization for PBM / healthcare legacy systems

Suggested closing line

“This is not a big-bang rewrite — it is a governed modernization factory.”